

Joshua Propfe

Hw4 midterm prep

1. Prompt: Please generate 3 essay questions related to deep neural networks, loss functions, and convolutional neural networks. Please base the questions on the slides provided. These slides are the material we have covered on those topics. Only provide me with questions. I will answer on my own and then get feedback on how I can answer better. This is a way for me to study for the midterm. Do not provide answers or feedback until I say I am ready for them. *I also attached the slides for those topics*

Response:

Question 1: Explain what happens mathematically when you compose two shallow networks into one deep network. Use the "folding" analogy to explain why the composed network can produce more linear regions than a shallow network with a similar number of parameters.

Question 2: Apply the recipe to binary and multiclass classification. Explain why sigmoid and softmax functions are needed, and how the model's distributional output becomes a single point prediction at inference.

Question 3: Explain how stride, dilation, kernel size, and depth affect the receptive field and the number of parameters. Show how to count parameters for a layer with C_i input channels, C_o output channels, and a $K \times K$ kernel.

AI tool used: Claude Opus 5.5

2. Question 1: When you compose two shallow networks, rather than producing an output directly, you are feeding that network as input into another network or hidden layer. Using more than one hidden layer is what makes the network deep as opposed to shallow. Each layer can make changes to the input, allowing the model to learn more complex patterns without adding more neurons. The folding analogy refers to the idea of a model like a piece of paper. Each additional hidden layer adds a fold to the paper and creates more linear regions. More regions allow for the network to behave differently and represent more complicated functions. More depth in the network allows for more complexity with a similar number of parameters.

Question 2: The recipe for loss functions follows these steps: first, choose the appropriate probability distribution. For binary, a Bernoulli distribution [0 or 1]. For Multiclass, a categorical distribution. Second, you set the model to predict the parameters. Third, you train the model and find the network parameters that minimize the negative log likelihood over the training data. Last, you perform a test and see how the model makes an inference. This functionally looks like the predicted class.

Networks produce a logit, which indicates how likely an input is to belong to a certain class. In Binary classification there are only 2 choices, so a sigmoid function is used to turn the logit into a number between 0 and 1. Softmax is used when there is more than two classes. It allows the network to produce 1 number for each class and for the sum of the numbers to add up to 1. This provides a probability for each individual class, with the highest number representing the highest probability classification. This becomes a single number because it outputs only the probability for the selected class in both binary and multiclass.

Question 3: The receptive field represents the portion of the input that the neuron in a model can see. This “field of vision” can be manipulated and controlled by the model engineer in a few ways; kernel size, stride, and dilation. Kernel size influences the number of inputs the neuron can see at any given time. Stride controls how far the kernel moves each time, for example does the field of vision move over one pixel at a time or does it move 3 pixels at a time. Increased dilation feeds the inputs into less neurons, this allows the kernel to cover more with less parameters. Adding more depth will also increase the receptive field. Each hidden layer builds on the information from the one before it, this allows the model to see more of the input than the original kernel size input.

To calculate the number of parameters in a convolutional layer you will multiply the number of inputs (C_i) by the number of outputs (C_o) and kernel size squared (K^2). For example, a model with $C_i=4$, $C_o= 10$, and $K=3$ would be $4 \times 10 \times 9 = 360$.

3. Prompt: Here are the responses to the questions. Please critique my answers. (I pasted in my responses from above.)
4. Adjusted Responses:

Question 1: When you compose two shallow networks, rather than producing an output directly, the second network's hidden-unit pre-activations are linear combinations of the first network's hidden-unit activations. The combined model becomes a piecewise-linear function applied to a piecewise-linear function. Using more than one hidden layer is what makes the network deep as opposed to shallow. Each layer can make changes to the input, allowing the model to learn more complex patterns without adding more neurons. The folding analogy refers to the idea of a model like a piece of paper. The first network folds the input space onto itself, so several input regions map to the same output range. The second network's function is then applied to every folded copy at once. That is why regions multiply rather than add. More regions allow for the network to behave differently and represent more complicated functions. More depth in the network allows for more complexity with a similar number of parameters.

Question 2: The recipe for loss functions follows these steps: first, choose the appropriate probability distribution. For binary, a Bernoulli distribution $\{0,1\}$. For Multiclass, a categorical distribution. Second, you set the model to predict the parameters. Third, you train the model and find the network parameters that minimize the negative log likelihood over the training data. Last, you perform a test and you take the maximum of the predicted distribution. This functionally looks like the predicted class.

Networks produce a logit but the network output can be any real number, while λ must lie in $[0,1]$ (and for multiclass, the λ_k must sum to 1). Without that constraint, the model isn't predicting a valid distribution. In Binary classification there are only 2 choices, so a sigmoid function is used to turn the logit into a number between 0 and 1. Softmax is used when there is more than two classes. It allows the network to produce 1 number for each class and for the sum of the numbers to add up to 1. This provides a probability for each individual class, with the highest number representing the highest probability classification. This becomes a single number because in binary the model predicts $y = 1$ if $\lambda > 0.5$, otherwise 0 and with Multiclass the model predicts the class with the highest probability (argmax).

Question 3: The receptive field represents the portion of the input that the neuron in a model can see. This "field of vision" can be manipulated and controlled by the model engineer in a few ways; kernel size, stride, and dilation. Kernel size influences the number of inputs, the neuron can see at any given time. Stride controls how far

the kernel moves each time, for example does the field of vision move over one pixel at a time or does it move 3 pixels at a time. It shrinks the output, which makes receptive fields grow faster in later layers. Dilation spaces the kernel weights apart, inserting zeros between them. For example, 3 weights with dilation 2 span 5 inputs. The receptive field widens and the parameter count stays the same. Adding more depth will also increase the receptive field with less cost than kernel size. Each hidden layer builds on the information from the one before it, this allows the model to see more input channels than the original kernel size input.

To calculate the number of parameters in a convolutional layer you will multiply the number of inputs (C_i) by the number of outputs (C_o) and kernel size squared (K^2) plus the bias C_o .

For example, a model with $C_i=4$, $C_o= 10$, and $K=3$ would be $4 \times 10 \times 9 + 10 = 370$.

Prompt after updated answers: I edited the responses based on your critique what gaps am I still missing? (copied above updated responses).

Claude Response:

Question 1

- "Without adding more neurons" It's inaccurate, because a new layer adds hidden units. The real point is more regions per parameter.
- Add the slide comparison: the shallow network has 19 parameters and at most 7 regions, while the composed network has 20 parameters and at least 9 regions. That one sentence is what makes "regions multiply" concrete.
- The regions are mirrored copies with dependencies between them, not independent pieces.
- Composing and combining are conflated. Composing means feeding network 1's output y in as network 2's input. Combining means substituting one into the other to get a single two-layer network. Your first sentence jumps straight to the second.
- The metaphor is muddled. The input space is the paper, not the model.

Question 2

- Sigmoid and softmax belong inside step 2. Step 2 is where the network predicts the distribution's parameters, and that's exactly where the $[0,1]$ constraint applies. You currently explain them separately, so the recipe and the explanation don't connect.
- The softmax mechanism is missing. Say how it works: exponentiate each output so it's positive, then divide by the sum so they total 1.
- Why sigmoid fits the binary case: Bernoulli has one parameter (λ), so the network needs only one output. "Only 2 choices" is less precise.

- "Perform a test" is wrong framing. Inference is simply how you make a prediction for a new input; it isn't a test.
- Missing the names `binary.cross_entropy` and `multiclass.cross_entropy`, and the loss equation itself.
- Minor notation: "a Bernoulli distribution $\{0,1\}$ " should be "a Bernoulli distribution over outcomes $y \in \{0,1\}$."

Question 3

- The parameter formula still says "inputs" and "outputs." These should be input and output channels.
- Still missing the key point: the parameter count does not depend on image size. That's the whole payoff of parameter sharing.
- No quantified growth. State that stacked 3-wide, stride-1 layers give receptive fields of 3, 5, 7, and so on.
- Parameter effects are incomplete. State that kernel size costs parameters (they grow with K^2) and that stride adds none.
- Dilation tradeoff missing. Dilation skips inputs in the gaps between weights, so it trades coverage density for reach.

What Changed:

Question 1: On Question 1 the initial feedback was more general. It corrected major ideas I misrepresented. It gave me feedback on the terminology I was using to describe composing networks and on how my initial folding analogy did not represent the multiplicative aspect of composing networks. In the second critique the feedback focused more on specific phrasing that was incorrect or misleading, and added that I should include how layers are mirrored copies, not independent.

Question 2: In the first critique it provided feedback on the major errors I made. I described point prediction incorrectly, It wanted me to explain why sigmoid and softmaxing are used/ need. Bigger scale errors. In the second critique, the feedback became much pickier about the things I could have also included to explain topics in more detail; sigmoid function, softmax, binary cross entropy loss, multiclass cross entropy loss.

Question 3: Like the first two, the initial feedback was focused primarily on full blown errors. I forgot to add bias b_0 to the formula, I described dilation incorrectly, I gave only partial answer to stride. The second critique focused less on what I got wrong, but the ideas I was missing and connections I wasn't making. Most of the

suggestions were synthesis based and asking for better understanding of why and how KS, stride, dilation, and depth relate to model architecture.

5. This was an interesting exercise. I have used AI to study for exams or tests before but have not used this method. What I found most interesting was the change in feedback in the second critique. I would not generally ask for a critique a second time but there was indeed an increased depth of feedback. I think this helped first identify information I did and didn't know and then took it a step further by helping me to understand where my understanding was somewhat shallow or underdeveloped. Beyond my awareness of what I do and do not understand correctly, this was valuable review. I think this type of review is more useful than other methods I have used in the past because it forces me to produce information and then fix what's wrong, which reinforces the content.

AI use statement: Claude opus 5.5 was used for this assignment. Review questions were generated, then my independent responses were critiqued using AI two times.